Shahjalal University of Science and Technology
Department of Computer Science & Engineering

Object-Oriented Programming Language (Java)

CSE 233 (0613 2133)

Comprehensive Question Bank

Organised by Curriculum Topic

Exam Coverage: CSE 16 – CSE 23

Note. This bank is a curated, topic-wise compilation of past year questions from SUST CSE for the course above. Questions are reorganised by curriculum topic for focused study. Marks and exam-batch context are preserved alongside each question.

Compiled for self-study by Koushik Sadhak Jeet

Contents

1 Introduction to Java **2**
2 Data Types, Variables, Arrays, Strings **3**
3 Operators & Expressions **7**
4 Control Statements **10**
5 Classes, Objects, Methods **13**
6 Inheritance **20**
7 Packages & Interfaces **34**
8 Exception Handling **39**
9 Multithreaded Programming **45**
10 Enumerations, I/O, Generics **47**
11 Lambda Expressions, Modules, Records **52**
12 String Handling & Collections Framework **53**
13 AWT, Swing, Event Handling **53**

1. Introduction to Java

Q1. JVM is platform independent. Answer YES / NO. **[2]** *[CSE 23, Quiz A]*

Q2. Full form of JRE. **[2]** *[CSE 23, Quiz B]*

Q3. Which one can compile java code: JVM or JRE or JDK? **[2]** *[CSE 23, Quiz C]*

Q4. JIT compiles what? **[2]** *[CSE 23, Quiz D]*

Q5. What is Java *HotSpot* technology? *[CSE 20, Final; CSE 21, Final]* **[2]** *[CSE 20, Final]*

Q6. What is *Applet* and *Servlet*? **[2]** *[CSE 20, Final]*

Q7. How many Java Platforms are there? Write their names. **[2]** *[CSE 20, Final]*

Q8. What are bytecode and sandbox? **[2]** *[CSE 21, Final]*

Q9. Mention the key implementation that helped Java make it independent of platforms? What is its impact on performance in case of speed issues? **[2]** *[CSE 19, Final]*

Q10. What gives Java its "write once and run anywhere" nature? (Also asked as: What gives Java the Platform independence property?)**[2]** *[CSE 19, Final; CSE 20, Final; CSE 21, Final]*

Q11. What is the relation between JDK, JRE, and JVM? **[2]** *[2017-18 Akash]*

Q12. Draw a block diagram to illustrate the relationships among JVM, JRE, and JDK. **[2]** *[CSE 19, Final]*

Q13. Explain the function and distinguish between JDK, JRE, and JVM? **[5]** *[CSE 21, Final]*

Q14. Define True/False. If false give the correct answer.

a. Although Java bytecode differs from platform to platform, JVM is always the same on all platforms.
b. For Java, an int is 32 bits in a 32-bit operating system, and 64 bits in a 64-bit operating system.
c. In Java, `<<` is a bitwise operator that shifts left with zero fill.
d. JRE is a part of JVM, and JVM is a part of JDK.
e. `synchronize` and `$_$` are two valid identifiers in Java.

[5] *[CSE 20, TT1]*

Q15. Provide two differences between procedural and object oriented programming? What if main method is written as "static public void" instead of "public static void"? **[3]** *[CSE 16-17, Final]*

Q16. Despite being a popular language, what is the limitation of C language? **[2]** *[CSE 16-17, Final]*

Q17. Write the basic difference between the concepts of Object-Oriented Programming and Structured Programming. **[2]** *[CSE 19, TT1]*

Q18. Why is the main method static? [2] *[CSE 19, TT1]*

Q19. Explain why `main()` is declared as `public` and `static` in a Java Program. Is the following code correct? Explain.

```
import java.lang.Math;
// inside class
double test = Math.PI * 5;
```

[2] *[CSE 19, Final]*

2. Data Types, Variables, Arrays, Strings

Q1. What is the result type of the expression: `byte + short`? [2] *[CSE 23, Quiz A]*

Q2. What is the default value of an object reference instance variable? [2] *[CSE 23, Quiz B]*

Q3. Does Java perform implicit narrowing conversion from `double` to `float`? Answer YES / NO. [2] *[CSE 23, Quiz B]*

Q4. What will be the output? `IO.println(10 + 20 + "Java");` [2] *[CSE 23, Quiz B]*

Q5. Is it valid: `var myArray = new int[10];` Answer YES / NO. *[CSE 23, Quiz B; CSE 21, Final]* [2] *[CSE 23, Quiz B]*

Q6. Is it valid: `var strArray = {"AB","BC","CD"};` Answer YES / NO.[2] *[CSE 23, Quiz C]*

Q7. What is the default value of the local variables in JAVA? [2] *[2017-18 Akash]*

Q8. "Strings are immutable" – what does that mean? [2] *[CSE 19, Final]*

Q9. Write a block of code to print all the elements of an array containing string type members using the for-each version of loops. [2] *[CSE 19, Final]*

Q10. How is a constant declared in Java? [2] *[CSE 19, Final]*

Q11.
```
int a = 13;
byte b;
b = a;
```
What type of error may this code create and why? Rewrite it to solve the issue.

[2] *[CSE 19, Final]*

Q12. Are these valid literals for Java: i) `0x__7af` ii) `2147483647` [1] *[CSE 23, TT1]*

Q13. Are these valid identifiers for Java: i) `ennum` ii) `v.m` [1] *[CSE 23, TT1]*

Q14. Are these valid literals for Java: i) `0___0_0000` ii) `0xFFFFFFFFFFFFFFFL` iii) `123_456._789_123` iv) `'\u432'` [2] *[CSE 21, TT1]*

Q15. Are these valid identifiers for Java: i) `extend` ii) `_123$` iii) `main` iv) `ABC-F` [2] *[CSE*

21, TT1]

Q16. Are these valid literals for Java: i) `'\00'` ii) `0x121ab.4abp2f` **[1]** *[CSE 22, TT1]*

Q17. Are these valid identifiers for Java: i) `native` ii) `_$_` **[1]** *[CSE 22, TT1]*

Q18. Are these valid integer literals for Java:
i) `0_0_2` ii) `0x000` iii) `0101010b` iv) `123456789L` **[2]** *[CSE 20, Final]*

Q19. Are these valid floating point literals for Java:
i) `00.332` ii) `3.141E+12` iii) `0x6.35E+7` iv) `12_03_.14_16` **[2]** *[CSE 20, Final]*

Q20. Are these valid identifiers for Java:
(CSE 20) i) `synchronized` ii) `$1115` iii) `?implement` iv) `value#32`
(CSE 21) i) `finally` ii) `$50` iii) `e-value` iv) `data%20` **[2]** *[CSE 20, Final; CSE 21, Final]*

Q21. Write valid/invalid for the following literals and identifiers in Java.
Literals: i) `0b4aBe` ii) `0454.3` iii) `123.45_F` iv) `'123'` v) `0x12.fep2_2`
Identifiers: vi) `_i_` vii) `Class` viii) `catch` ix) `100$` x) `permit` **[5]** *[CSE 22, Final]*

Q22. Write only valid/invalid for the following literals and identifiers in Java.
Literals: i) `0_0_0_0_0f` ii) `07281` iii) `'\u033'` iv) `0xfACC` v) `0x1.fp-3`
Identifiers: vi) `n_ortc` vii) `yield` viii) `instanceof` ix) `var=n` x) `!main` **[5]** *[CSE 23, Final]*

Q23. For the following literals, determine valid or invalid. If valid, write the data type:

```
i. 99.5              vi. 9.9.5_3f
ii. 20L              vii. "22.38"
iii. 0b10_11         viii. 0x5.5e5
iv. 0b10l_11         ix. 0x5.5p5
v. 1_2_._3           x. _1_23
```

[10] *[CSE 16-17, Final]*

Q24. Define True/False. If false, give correct answer.

i. The range of character data type in Java is −32768 to +32767.
ii. `int i = 212L;` is a valid statement in Java.
iii. In Java, Double is a primitive data type.
iv. In Java, a two-dimensional array can be determined by length() method.
v. `int [ ] arr = {10, 20, 30};` is a valid statement in Java.

[5] *[CSE 16-17, Final]*

Q25. What is the best approach if frequent string concatenations performed? Write the difference between StringBuffer and StringBuilder class? **[3]** *[CSE 16-17, Final]*

Q26. Check the following Java program carefully. Find the compilation errors and specify the reasons.
(CSE 20 version)

```
01.  class ConversionExam {
```

```
02.      public static void main (String[] args) {
03.          byte b = 0;
04.          b += 1;
05.          b = b-1;
06.          boolean bn = true;
07.          bn = (boolean) b;
08.          float f = 123.35;
09.          f = f+2.3;
10.      }
11. }
```

(CSE 21 version)

```
01.  class ConversionExam {
02.      public static void main (String[] args) {
03.          byte b = 0;
04.          b += 1;
05.          b = b-1;
06.          float f = 89.53;
07.          f = f+5.0;
08.          char c = b;
09.      }
10. }
```

[4] *[CSE 20, TT1; CSE 21, TT1]*

Q27. Check the following Java program carefully. Find the compilation errors and specify the reasons.

```
01.  class ConversionExam {
02.      public static void main (String[] args) {
03.          byte b = 9;            // CSE 21 uses b = 9
04.          b += 1;
05.          b = b-1;
06.          boolean bn = true;     // CSE 21 uses bn = false
07.          bn = (boolean) b;      // CSE 21: bf = (boolean) b;
08.          float f = 123.35;      // CSE 21 uses f = 112.53
09.          f = f+2.3;             // CSE 21 uses f = f-3.2
10.      }
11. }
```

[5] *[CSE 20, Final; CSE 21, Final]*

Q28. Check the following Java program carefully. Find what type of compilation error occurred in which line and specify the reasons/explanations.

```
01.  class ConversionExam {
02.      public static void main(String[] args) {
03.          byte b1 = (byte) 0.0001;
04.          b1 += 1;
05.          b1 = b1-1;
06.          boolean bf = (boolean) false;
```

```
07.          bf = (boolean) b1;
08.          float f = 172.53;
09.          f = f + 4.3;
10.          char c = b1;     }
11. }
```

[5] *[CSE 22, Final]*

Q29. Check the following Java program carefully. Find the compilation errors and specify the reasons. (For each wrong error identification, 0.5 marks will be deducted.)

```
01.  public class CompileError {
02.      public static void main (String[] args) {
03.          byte b = 0B10; b++; b = b << 2;
04.          long l = 0xCAFEL; l = b%0b001;
05.          boolean b1 = (boolean) (true); b2| =b1;
06.          char c = 'A'; c = c + c;
07.          double d = .5_0; d = b1?d:+5_0;
08.      }
09. }
```

[4] *[CSE 23, TT1]*

Q30. Check the following Java program carefully. Find the compilation errors and specify the reasons.

```
01. public class CompileErrors {
02.     public static void main (String[] args) {
03.         int i; i++;
04.         byte b = 0b00111011;
05.         b += 1;
06.         b = b-1;
07.         float f = 0x121ab.4adb2f;
08.         boolean b1 = (boolean)(true|false);
09.         b1 = 0b101&0b100;
10.         char c = 0x2f_a; c += 06;
11.     }
12. }
```

[4] *[CSE 22, TT1]*

Q31. Write down the output of the following program segment.

```
int p[] = {10, 23, 2, 121, 232, 211};
for(int i=1; i<p.length; i++)
    if(p[i]%2==0)
        p[i] = p[i] + p[i-1];
for (int i=0; i<p.length; i++)
    System.out.println(p[i]);
```

[5] *[2017-18 Akash]*

Q32. Is the given code below alright? If not then correct the errors and write the output of the code.

```
int sum = 0;
int sums[ ] [ ] = new int[3][5];
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 5; j++)
        sums [i][j] = (i+1) * (j+1);
}
for (int x[ ] : sums) {
    for (int y : x) {
        System.out.println("Value is : " + y);
        sum += y;
    }
}
System.out.println("Summation : " + sum);
```

[5] *[CSE 19, Final]*

Q33. Write the output of the following program.

```
public class StringDemo {
  public static void main(String args[]) {
    String s1 = "Java World";
    String s2 = "Programming";
    System.out.println("" + s1.substring(5));
    System.out.println("" + s1);
    s1 = s1.substring(5);
    System.out.println("" + s1);
    System.out.println("" + s2.substring(5));
    System.out.println("" + s2);
    System.out.println("" + s2.length());
    System.out.println("" + s2.indexOf('a'));
    System.out.println("" + s2.equalsIgnoreCase(s2.trim()));
    System.out.println("" + s2.equals(s2.trim()));
    System.out.println("" + s2.charAt(3));
    s2 = s2.replace("a", "A");
    System.out.println("" + s2.toUpperCase().equals("JAVA"));
  }
}
```

[10] *[CSE 16-17, Final]*

3. Operators & Expressions

Q1. Which operator performs an unsigned right shift? **[2]** *[CSE 23, Quiz A]*

Q2. What will be printed?

```
int x = 5;
IO.println(x++ + ++x);
```

[2] *[CSE 23, Quiz A]*

Q3. What would be the output of `IO.println(0b101 + 0x10);` **[2]** *[CSE 23, Quiz A]*

Q4. What will be printed? `IO.println(5 & 3);` **[2]** *[CSE 23, Quiz C]*

Q5. What operator can be used to create an object dynamically? **[2]** *[CSE 23, Quiz C]*

Q6. What is the result of `!true || true && false`? **[2]** *[CSE 23, Quiz D]*

Q7. Write down the purpose of the operator ∼ and « in Java. **[2]** *[CSE 19, Final]*

Q8. "Redundant parentheses may degrade the performance of your program" – justify this statement. **[2]** *[CSE 19, Final]*

Q9. What is the difference between » and »> operators in Java? *[CSE 20, Final; CSE 19, TT1]***[2]** *[CSE 20, Final]*

Q10. How does Java represent negative numbers? Show the encoding and decoding process with the numbers between 63 and -63. **[2]** *[CSE 20, Final; CSE 21, Final]*

Q11. If both the access specifiers "default" and "public" make members and methods of a class accessible from outside the class then what is the difference between them? **[2]** *[CSE 19, Final]*

Q12. Consider the following Java program:

```
01.  class OpExam {
02.      public static void main (String[] args) {
03.          int a = 135, b = 56, c = 71;
04.          a = ab + c;
05.          System.out.println(a + " " + b + " " + c);
06.          a = 21; b = 7;
07.          System.out.println(a + " " + b + " " + (a&b));
08.          byte i = -1; a = i << 24;
09.          System.out.println(i + " " + 24 + " " + a);
10.          a = i << 24; System.out.println(i + " " + 24 + " " + a);
11.      }
12. }
```

Fill the _ with suitable operators that will output as follows (do not write the full program, write only those statements with line numbers that are affected):

```
126 55 72
21 _ 7 = 18
-1 _ 24 = -1
-1 _ 24 = 255
```

[5] *[CSE 20, TT1]*

Q13. Consider the following Java program:

```
01.  public class OpExam {
```

```
02.      public static void main (String[] args) {
03.          int a = 147, b = 73, c = 39;
04.          a = a_b + c_b + c;
05.          System.out.println(a + " " + b + " " + c);
06.          a = 37; b = 19;
07.          System.out.println(a + " " + b + " " + (a&b));
08.          b = -11; a = b _ 4;
09.          System.out.println(b + " " + _ 4 + " " + a);
10.          a = b _ 26; System.out.println(b + " " + _ 26 + " " + a);
11.      }
12. }
```

Fill the _ with suitable operators that will output as follows (do not write the full program; write only those statements with line numbers that are affected):

```
114 74 38
37 _ 19 = 54
-11 _ 4 = -1
-11 _ 26 = 63
```

[6] *[CSE 21, TT1]*

Q14. Consider the following Java program:

```
01.  class OpExam {
02.     public static void main (String[] args) {
03.          int a = 125, b = 76, c = 31;
04.          a = (b + c);
05.          System.out.println(a + " " + b + " " + c);
06.          a = 41; b = 9;
07.          System.out.println(a + " " + b + " " + (a&b));
08.          byte i = -10; a = i << 23;
09.          System.out.println(i + " " + 23 + " " + a);
10.          a = i << 20; System.out.println(i + " " + 20 + " " + a);
11.      }
12. }
```

Fill the x with suitable operators that will output as follows:

```
106 75 32
41 9 = 32
-10 23 = -1
-10 20 = 4095
```

[5] *[CSE 20, Final]*

Q15. Consider the following Java program:

```
01.  public class OpExam {
02.      public static void main(String[] args) {
03.          int a = 125, b = 76, c = 31;
04.          x = a^b ? 0 + (++c) : b;
```

```
05.          System.out.println(a + " " + b + " " + c);
06.          a = 41; b = 9;
07.          System.out.println(a + " " + b + " " + (a^b));
08.          byte i = 10; a = i.<< 3;
09.          System.out.println(i + " " + 3 + " " + a);
10.          System.out.println("" + a + " " + a);
11.      }
12. }
```

Fill the `x` with suitable operators that will output as follows:

```
99 75 32
41   9 = 9
10   3 = 80
80 = 81
```

[5] *[CSE 21, Final]*

Q16. Show the difference between » (Shift right) and »> (Shift right zero fill) operators using an example. **[2]** *[CSE 19, TT1]*

Q17. Check the following java program carefully. Do you think there is any compilation error? If there is find the errors, correct it using line number and then write the output. For correcting you cannot insert new statement or delete a statement. If there is no error then write only output.

```
01 public class testJava {
02   public static void main (String[] args) {
03     int a = 517, b = 65, c = 27;
04     a -= b - c++;
05     System.out.println("1" + a + " " + b + " " + c);
06     a = 21; b = 19;
07     System.out.println("2" + (a*b));
08     System.out.println("3" + (a/b));
09     System.out.println("4" + (a%b));
10     byte i = 8; e = i + 5;
11     System.out.println("5" + e);
12     a = 19; b = 5; System.out.println("6" + a);
13     a = 87; b = 9; System.out.println("7" + a);
14     boolean d = true;
15     d = a > System.out.println("8" + d);
16     d = a < System.out.println("9" + d);
17     d = a >= System.out.println("10" + d);
18     System.out.println("11" + ((a==e) && (a < b) ? "Close" : "End"));
19   }
20 }
```

[10] *[CSE 16-17, Final]*

4. Control Statements

Q1. Which special version of **for loop** is found in Java? Explain its use with a suitable example. **[2.5]** *[CSE 19, TT1]*

Q2. What kind of extra facility can we get while using **break** in a nested loop in Java which was not available in C language? **[2.5]** *[CSE 19, TT1]*

Q3. Write the output of the following Java program.

```
01.  class IterationExam {
02.      public static void main(String[] args) {
03.          int i = 0;
04.          First: while (i <= 6) {
05.              int sum = 0; i++;
06.              Second: for (int j = 1; j <= i;) {
07.                  sum = sum + j++;
08.                  for (int k = 1; k <= j; k++) {
09.                      if (k%2==0) continue;
10.                      sum = sum + k;
11.                      if (sum>50) break Second;
12.                  }
13.                  if (i == 3) continue First; }
14.              System.out.print(sum + " ");
15.          }
16.      }
17. }
```

[6] *[CSE 20, TT1]*

Q4. Write the output of the following Java program.

```
01.  class IterationExam {
02.      public static void main(String[] args) {
03.          int i = 0;
04.          B1: while (i <= 6) {
05.              int sum = 0; i++;
06.              B2: for (int j = 1, k=1; j <= i;) {
07.                  sum = sum + j++;
08.                  for (; k <= j; k++) {
09.                      if (k%2==0) continue;
10.                      sum = sum + k;
11.                      if (sum>50) break B2;
12.                  }
13.                  if (i == 3) continue B1; }
14.              System.out.println(sum);
15.          }
16.      }
17. }
```

[6] *[CSE 21, TT1]*

Q5. Write the output of the following Java program.

```
01.  class IterationExam {
02.      public static void main(String[] args) {
03.          int i = 2;
04.          First: while (i <= 9) {
05.              int sum = 100;     i++;
06.              Second: for (int j = 1; j < i; j++) {
07.                  sum = sum + j*i;
08.                  for (int k = 0; k <= j; k+=3) {
09.                      if (k%3!=k-k) continue;
10.                      sum = sum + k;
11.                      if (sum>200) break Second;
12.                  }
13.                  if (i == 3) continue First; }
14.              System.out.print(sum + " ");
15.          }
16.      }
17. }
```

[5] *[CSE 20, Final]*

Q6. Write the output of the following Java program.

```
01.  public class ControlExam {
02.      public static void main(String[] args) { int i = 0;
03.          B1: for (;i<=10;i+=2) {
04.              int sum = i-i;
05.              B2: for (int j = 1, k=4; j<=i; j+=3) {
06.                  sum= sum + j++;
07.                  for (; k<5; k++) {
08.                      if (j%5==0) break B1; sum++;
09.                      if (i+sum>i*2) continue B2;
10.                  }
11.                  if (j%2==0) {i++;break;}
12.              IO.println(i+" "+sum);
13.              }
14.          }
15.      }
16. }
```

[4] *[CSE 23, TT1]*

Q7. Write the output of the following Java program.

```
01. public class ControlExam {
02.     public static void main(String[] args) { int i = 0;
03.         B1: while (i <= 10) {
04.             int sum = 0;i++;++i;
05.             B2: for (int j = 1,k=1; j <= i;) {
06.                 sum = sum + j++;
07.                 for (; k<5; k++) {
08.                     if (j%2!=0) continue B1;
```

```
09.                     sum = sum + k;
10.                     if (i+sum>i*3) break B2;
11.                 }
12.                 if (j%5==0) { i++;break;} }
13.             System.out.println(i+" "+sum);
14.         }
15.     }
16. }
```

[4] *[CSE 22, TT1]*

5. Classes, Objects, Methods

Q1. Does constructor execution happen before non-static initialization blocks? Answer YES / NO. [2] *[CSE 23, Quiz A]*

Q2. Is it possible to create an abstract class without any abstract methods? Answer YES / NO. [2] *[CSE 23, Quiz A]*

Q3. What do we call a non-static nested class? [2] *[CSE 23, Quiz B]*

Q4. Is it possible to declare a `private` method `final`? Answer YES / NO. [2] *[CSE 23, Quiz B]*

Q5. How can we force a sub-class to override a method? [2] *[CSE 23, Quiz C]*

Q6. Can we create private constructor in a class? Answer YES / NO. [2] *[CSE 23, Quiz C]*

Q7. Can a static method access `this`? Answer YES / NO. [2] *[CSE 23, Quiz C]*

Q8. Static nested class cannot be extended! Answer YES / NO. [2] *[CSE 23, Quiz D]*

Q9. `class A {class B }` – Create object of class `B`. [2] *[CSE 23, Quiz D]*

Q10. Can we use static variables in an instance method? Answer YES / NO. [2] *[CSE 23, Quiz D]*

Q11. Determine whether the following statement is true or false and explain your answer: "Constructors can be either static or non-static." [2] *[2017-18 Akash]*

Q12. Can we assign any reference to "this" variable? Explain your answer. [2] *[2017-18 Akash]*

Q13. What is the purpose of declaring a variable or method as final? [2] *[2017-18 Akash]*

Q14. What happens when the constructor of a class is declared as "private"? [2] *[2017-18 Akash; CSE 17, Final; CSE 18, TT]*

Q15. What are the restrictions that are applied to the Java static methods? [2] *[2017-18 Akash]*

Q16. What do you think the following fragment of code does?

```
1.  MyClass c = new MyClass();
```

```
2.  MyClass c2 = c;
3.  MyClass c3 = new MyClass();
```

[2] *[2017-18 Akash]*

Q17. When does garbage collector in Java works? Write three restrictions of static methods.[2] *[CSE 16-17, Final]*

Q18. Write down the usage and restriction of `this` in a class. [2] *[CSE 16-17, Final]*

Q19. What is the return type of a constructor? How can we call the constructor of the superclass from its subclass? [2] *[CSE 19, Final]*

Q20. Clarify the relationship between class and object. Mention the operator that plays a key role in this relationship? [2] *[CSE 19, Final]*

Q21. What does it mean when a class is declared final? What happens if an abstract class is declared final? [2] *[CSE 19, Final]*

Q22. What type of class can be declared as private? What is the issue with declaring a class as private? [2] *[CSE 19, Final]*

Q23. Define singleton class. [1] *[CSE 19, Final]*

Q24. What is the purpose of "constructor"? Why does a constructor not have any return type?[2] *[CSE 19, TT1]*

Q25. "Class is the template of an object" — explain in brief. [3] *[CSE 19, TT1]*

Q26. Which principle(s) of OOP is fulfilled through the usage of Class in Java? Explain in brief. [3] *[CSE 19, TT1]*

Q27. Can a class be defined as private? [1] *[CSE 19, TT1]*

Q28. Is it possible to create a private constructor or inherit a constructor? [2] *[CSE 20, Final; CSE 21, Final]*

Q29. How can we prevent a class from being inherited or a method from being overridden? [2] *[CSE 20, Final; CSE 21, Final]*

Q30. Can we overload the `main()` method? Explain. [2] *[CSE 21, Final]*

Q31. How can you prevent a method from being overridden? [2] *[CSE 21, Final]*

Q32. What are the default constructor and unnamed package in Java? [2] *[CSE 20, Final; CSE 21, Final]*

Q33. When is the finalize() method called in Java? [2] *[CSE 17, Final; CSE 18, TT]*

Q34. What is the purpose of declaring a class or method as final?[2] *[CSE 17, Final; CSE 18, TT]*

Q35. Differentiate between Call by Value and Call by Reference.[2] *[CSE 17, Final; CSE 18, TT]*

Q36. Define abstract class. What are the criteria for one class to extend its abstract class? Present a scenario that justifies the use of an abstract class. **[2+1+2]** **[3]** *[CSE 19, Final]*

Q37. Create a class `Account` with two overloaded constructors. The first constructor is used for initialising the name of the account holder, the account number, and the initial amount in the account. The second constructor is used for initialising the name of the account holder, the account number, addresses, the type of account, and the current balance. The Account class has methods `deposit()`, `withdraw()` and `getBalance()`. Make the necessary assumption for data members and return types of the methods. Create objects of Account class and use them. **[5]** *[CSE 19, Final]*

Q38. Define the `Invoice` class for the following program.

```
class Main{
    public static void main (String[] args) {
        Invoice inv = new Invoice("2034", 2, 325.50);
        System.out.println(inv.getPartNo() + " " + inv.getQuantity() + " "
                           + inv.printPerItem());
    }
}
```

[5] *[CSE 19, Final]*

Q39. What is the difference between Listener and Adapters in Java? Which one is better? **[2]** *[CSE 16-17, Final]*

Q40. Write down the purposes of the keywords and methods given below:

i. synchronized
ii. protected
iii. finally
iv. notify()
v. static

[5] *[CSE 19, Final]*

Q41. Despite being a popular language, what is the limitation of C language? Write on `final` variable, `final` method and `final` class with example. **[5]** *[CSE 16-17, Final]*

Q42. Define the class `Point`, `Shape`, `Rectangle` and `Circle` for the following program. All member variables of these classes should be private.

```
01 public class ObjectInClass {
02   public static void main(String[] args) {
03     Point p = new Point(1,1);
04     System.out.println("Point: "+p.getX()+", "+p.getY());
05     Rectangle r = new Rectangle(p, 20, 0, 60, 0);
06     System.out.println("R: "+r.getHeight()+", "+r.getWidth());
07     System.out.println("R-Area: "+r.area());
08     r.setHeight(40.0);  r.setWidth(80.0);
09     Shape s = r;
10     System.out.println("S-Point: "+s.getPoint().getX()
11                        +", "+s.getPoint().getY());
```

```
12     p.setX(1);  p.setY(2);
13     Circle c = new Circle(p,5,0);
14     System.out.println("C: "+c.getRadius()+", "+c.Area("c.area()));
15     c.setRadius(4.0);
16     s = c;     System.out.println("S-Area: "+s.area());
17   }
18 }
```

```
Output of this program:
Point: 1, 2
R: 60.0, 20.0
R-Area: 800.0
S-Area: 60.0
S-Point: 1, 2
C: 5.0, C-Area: 78.5
S-Area: 50.24
```

[10] *[2017-18 Akash]*

Q43. Define the class `Point`, `Shape`, `Rectangle` and `Circle` for the following program. All member variables of these classes should be private.

```
01 public class ObjectToClass {
02   public static void main (String[] args) {
03     Point p = new Point(1,2);
04     System.out.println("Point: "+p.getX()+", "+p.getY());
05     Rectangle r = new Rectangle(p, 20, 0, 40, 0);
06     System.out.println("R: "+r.getHeight()+", "+r.getWidth());
07     System.out.println("R-Area: "+r.area());
08     r.setHeight(8.0); r.setWidth(6.0);
09     Shape s = r; System.out.println("S-Point: "+s.getPoint().getX()
10                           +", "+s.getPoint().getY());
11     p.setX(1); p.setY(0);
12     Circle c = new Circle(p,5,0);
13     System.out.println("C: "+c.getRadius()+", C-Area: "+c.area());
14     c.setRadius(4.0);
15     s = c; System.out.println("S-Area: "+s.area());
16   }
17 }
```

```
Output of this program:
Point: 1,2
R: 40.0, 20.0
R-Area: 800.0
S-Area: 48.0
S-Point: 1,2
C: 5.0, C-Area: 78.5
S-Area: 50.24
```

[10] *[CSE 16-17, Final]*

Q44. `public class MyClass {`

```
    int a = 3, b;
    void meth(int x) {
        System.out.println("x = " + x + "a = " + a + "b = " + b);
    }
    static {
        System.out.println("Static block initialized.");
    }
    int a = 4;
    public static void main(String args[]){
        meth(42);
    }
}
```

Write the output of the given code. If you think it has bug(s) then rewrite it, fixing the bug and mentioning the causes.

[5] *[CSE 19, Final]*

Q45. Consider the following Java programs.

```
01.  class BlockExam {
02.      { System.out.println("B1: First Block"); }
03.      int a; static int s = 5;
04.      BlockExam(int a) {        this.a = a;
05.          System.out.println("B3: Constructor; a = "+a+",s = "+s);}
06.      { a = 22; s = 33; }
07.      { System.out.println("B3: a = "+a+",s = "+s); }
08.      { s += 55; }
09.      static { System.out.println("B4: a = "+a); }
10.      public static void main(String args[]) {
11.          System.out.println("Starting Exam!!");
12.          new BlockExam(74); s++;
13.          { System.out.println("B5: a = "+a+",s = "+s); s++;}
14.          static { System.out.println("B6: s = "+s); }
15.      }
16. }
```

[10] *[CSE 20, Final]*

Q46. Write the output of the following Java program.

```
01.  class BlockExam {
02.      { System.out.println("B1: First Block"); }
03.      int a; static int s = 5;
04.      BlockExam(int a) {        this.a = a;
05.          System.out.println("B3: Constructor; a = "+a+",s = "+s);}
06.      { a = 20; s = 30; }
07.      { System.out.println("B3: a = "+a+",s = "+s); }
08.      { s += 50; }
09.      static { System.out.println("B4: a = "+a); }
10.      public static void main(String args[]) {
11.          System.out.println("Starting Exam!!");
12.          new BlockExam(50); s++;
```

```
13.          { System.out.println("B5: a = "+a+",s = "+s); s++;}
14.          static { System.out.println("B6: s = "+s); }
15.      }
16. }
```

[10] *[CSE 21, Final]*

Q47. Check the following Java program carefully. Is there any compilation error? If there are, find the errors, correct them using the line number, and then write the output. You cannot insert a new statement or delete a statement to correct. If there is no error, then write only the output.

```
01.  class BlockExam {
02.      { System.out.println("B1: a = "+a); }
03.      int a = 11; static int s = 12;
04.      BlockExam(int a) {      this.a = a;
05.          System.out.println("B2: a = "+a+",s = "+s); }
06.      { a = 13; s = 14; }
07.      { System.out.println("B3: a = "+a+",s = "+s); }
08.      { s += 15; }
09.      static { System.out.println("B4: a = "+a); }
10.      public static void main(String args[]) {
11.          System.out.println("Main: BlockExam Started!!");
12.          new BlockExam(16); new BlockExam(17);
13.      }
14.      { System.out.println("B5: a = "+a+",s = "+s); s++;}
15.      static { System.out.println("B6: s = "+s); }
16. }
```

[10] *[CSE 22, Final]*

Q48. Check the following Java program carefully. Is there any compilation error? If so, identify the error using the line number and provide the correction. Finally, write the output of the program.
Note: You cannot insert or delete statements to fix errors.

```
1.  class BlockExam {
2.      { 10.println("B1: a = "+a); }
3.      int a = 50; static int s = 60;
4.      { a = 30; s = 40; }
5.      { 10.println("B2: a = "+a+",s = "+s); }
6.      static class SC { static int s = 70; }
7.      static { 10.println("B3: s = "+s); }
8.      { s += 5; }
9.      static { 10.println("B4: a = "+a); }
10.     public static void main(String args[]) {
11.         10.println("Main: BlockExam Started!");
12.         new BlockExam(10); new BlockExam(20);
13.     }
14.     { 10.println("B5: a = "+a+",s = "+s); s++; }
15.     BlockExam(int a) { a = a;
```

```
16.         10.println("B6: a = "+a+",s = "+s); }
17.     static { 10.println("B7: s = "+s); }
18. }
```

[10] *[CSE 23, Final]*

Q49. Determine the following code is correct or incorrect. If the code is correct simply write down the output, if incorrect, specify why and rewrite the correct code.

```
1.  class A {
2.    static void staticMethod() {
3.      System.out.println("Static Method");
4.    }
5.  }
6.  public class MainClass {
7.    public static void main(String[] args) {
8.      A a = null;
9.      a.staticMethod();
10.   }
11. }
```

[5] *[2017-18 Akash]*

Q50. Check the following java program carefully, you can insert as many as statements you want but you can't delete or edit any statement from MainClass.

```
1.  public class MainClass {
2.    public static void main(String[] args) {
3.      MySystem.out.println("Static Method");
4.    }
5.  }
```

[5] *[2017-18 Akash]*

Q51. What are the restrictions of generics? How generics improve type safety? (Note: portion about reverseArray belongs in Generics topic.) **[5]** *[2017-18 Akash]*

Q52. Briefly describe the garbage collection mechanism of JAVA. **[2]** *[CSE 16-17, Final]*

Q53. Consider the following code and determine at what point is the Bar object, created on line 6, eligible for garbage collection?

```
1.  class Bar { }
2.  }
3.  public class Test {
4.    Bar doBar() {
5.      Bar b = new Bar();
6.      return b;
7.    }
8.    public static void main (String args[]) {
9.      Test t = new Test();
10.     Bar newBar = t.doBar();
```

```
11.     System.out.println("newBar");
12.     newBar = new Bar();
13.     System.out.println("finishing");
14.   }
15. }
```

[3] *[CSE 16-17, Final; CSE 19, Final]*

Q54. What is the signature of a method? Define method overloading. Write a simple code that exemplifies method overloading. *[CSE 19, Final; CSE 23, Final]* [5] *[CSE 19, Final]*

Q55. What is meant by method overloading with type promotion? Show with a code snippet.[5] *[CSE 21, Final]*

Q56. Consider the class `Q1` provided below. Your task is to implement class `A` such that running the `main` method produces the specified output.
Restriction: Class `A` must not contain any print statements.

```
01. public class Q1 {
02.     public static void main(String[] args) {
03.         A a = new A("Hello"); IO.println(a);
04.         A b = new A("World"); IO.println(b);
05.     }
06. }
```

Expected Output:

```
HelloHello
WorldWorld
```

(Source corrected: this is CSE 23, TT2 – see also similar problem.)

[4] *[CSE 22, TT2]*

6. Inheritance

Q1. Is constructor overriding allowed in Java? Answer YES / NO. [2] *[CSE 23, Quiz D]*

Q2. By which mechanism call to an overridden method is resolved at runtime? [2] *[CSE 23, Quiz D]*

Q3. What are the purposes of super keyword? [2] *[2017-18 Akash]*

Q4. Can we change the scope of the overridden method in the subclass? [2] *[2017-18 Akash]*

Q5. Does Java support multiple inheritances? If not why? [2] *[2017-18 Akash]*

Q6. What happens if Java allows multiple inheritance? Show an example.[2] *[CSE 16-17, Final]*

Q7. What are the uses of "super" keyword. [2] *[CSE 17, Final; CSE 18, TT]*

Q8. Is it possible to have different return type for an overridden method? **[2]** *[CSE 17, Final; CSE 18, TT]*

Q9. Can we override an overloaded method? **[2]** *[CSE 17, Final; CSE 18, TT]*

Q10. Can you override a private method in Java? **[2]** *[CSE 17, Final; CSE 18, TT]*

Q11. What do you know about Dynamic Method Dispatch in Java? **[3]** *[CSE 16-17, Final]*

Q12. What is dynamic method dispatching? In which case does Java implement this concept? Which principle of OOP is supported through this implementation? Write a code as an example of this concept. **[2+2+1+3]** **[8]** *[CSE 19, TT2]*

Q13. Explain the reason for Java's not supporting multiple inheritances. How can you implement this concept in Java? What is the name of the thread that begins immediately when a Java program starts? **[2+2+1]** **[5]** *[CSE 19, Final]*

Q14. What is method overriding? What happens if we change the arguments of the overriding method? How do you refer to the superclass version of the overridden method in the sub-class? Can we override a private method? **[2+1+1+1]** **[5]** *[CSE 19, Final]*

Q15. What happens when a superclass variable refers to a subclass object? Explain the concept with suitable code blocks. **[1+4]** **[5]** *[CSE 19, Final]*

Q16. Present a discussion on Class, Abstract Class, and Interface. How do they defer from each other? What type of relationships are found among them? **[2+2]** **[4]** *[CSE 19, TT2]*

Q17. Check the following code carefully. Write the output of the following codes with explanation to your answer. If there is any error, specify it with reason.

```
class Animal {
  void jump() {
    System.out.println("Animal");
  }
  void jump(double a) {
    System.out.println("Animal:"+a);
  }
}
class Cat extends Animal {
  void jump(int a) {
    System.out.println("Cat");
  }
}
class Horse extends Animal {
  void jump(int a) {
    System.out.println("Horse");
    super.jump();
  }
}
public class Circus {
  public static void main(String args[]) {
```

```
    Animal cat = new Cat();
    Horse horse = new Horse();
    cat.jump();
    horse.jump(10,1);
    horse.jump();
  }
}
```

[12] *[CSE 16-17, Final]*

Q18. Write down the output of the following program:

```
abstract class A {
  A(int a) { System.out.println("A(int)"); }
  A() { System.out.println("A()"); }
  abstract void x();
  void y() {
    System.out.println("A.y()");
    x();
  }
}
class B extends A {
  void x() {
    System.out.println("B.x()");
  }
  void y() {
    System.out.println("B.y()");
  }
  void z() {
    System.out.println("B.z()");
    super.y(); y();
  }
}
public class TestDemo {
  public static void main(String[] args) {
    B b = new B(); b.z();
  }
}
```

[10] *[CSE 16-17, Final]*

Q19. Will the following program work properly? Does the program represent polymorphism property of Java and how?

```
public class Main {
    public static void main(String[] args) {
        main();
    }
    static void main() {
        System.out.println("It is main method");
    }
}
```

[5] *[2017-18 Akash]*

Q20. Explain the basic principles of OOP. **[2]** *[2017-18 Akash]*

Q21. What is method overloading and overriding in Java? **[2]** *[2017-18 Akash]*

Q22. Check the following code carefully.

```
public class Person {
    public void work(){
        system.out.println("Working");
    }
    public void sleep(){
        system.out.println("Sleeping");
    }
}
```

```
public class Teacher extends Person {
    public void work(){
        system.out.println("Teaching");
    }
    public void work(String course){
        system.out.println("Teaching
        courses");
    }
}
```

Now write the output of the following codes with the explanation to your answer. If there is an error, specify it with reason.

i. Person p = new Person();
ii. p.work();
iii. p.sleep();
iv. Teacher t = new Teacher();
v. work("OOP");

[8] *[CSE 19, Final]*

Q23. What is the output of the program and give the logic behind your answer.

```
class A{
    int i;
    void display(){
        System.out.println();
    }
}
class B extends A{
    int j;
    void display(){
        System.out.println();
    }
}
class Test{
    public static void main(String args[]){
        B obj = new B();    obj.i = 1;    obj.j = 2;
        obj.display();
    }
}
```

[5] *[CSE 19, Final]*

Q24. Is this the output of the following Java program? Explain the output.

```
01.  class One{
02.      public static void print() {
03.          System.out.println("1");
04.      }
05.      class Two extends One {
06.          public static void print() {
07.              System.out.println("2");
08.          }
09.          public class Test {
10.              public static void main (String args[]) {
11.                  One one = new Two();
12.                  one.print();
13.              }
14.          }
15.      }
16. }
```

[5] *[CSE 22, Final]*

Q25. What is the output of the following Java program? Explain the output.

```
01.  class Base {
02.      public Base () {
03.          System.out.println(" Base ");
04.      }
05. }
06.  class Derived extends Base {
07.      public Derived () {
08.          System.out.println(" Derived ");
09.      }
10. }
11.  class DeriDerived extends Derived {
12.      public DeriDerived() {
13.          System.out.println(" DeriDerived ");
14.      }
15. }
16.  public class Test {
17.      public static void main(String[] args) {
18.          Derived b = new DeriDerived();
19.      }
20. }
```

[5] *[CSE 22, Final]*

Q26. Write the output of the following Java program.

```
01.  class Automobile {
02.      private String drive() {
03.          return "Driving __ vehicle";
04.      }
05. }
```

```
06.  class Car extends Automobile {
07.      protected String drive() {
08.          return "Driving __ car";
09.      }
10. }
11.  public class ElectricCar extends Car {
12.      @Override
13.      public final String drive() {
14.          return "Driving __ an__ electric __ car";
15.      }
16.      public static void main(String[] wheels) {
17.          final Car car = new ElectricCar();
18.          System.out.print(car.drive());
19.      }
20. }
```

[5] *[CSE 22, Final]*

Q27. Analyze the following code carefully and answer the questions that follow.

```
public class Vehicle {
    public void start() {
        System.out.println("Vehicle starting...");
    }
}
public class Car extends Vehicle {
    public void start() {
        System.out.println("Car engine starting...");
    }
    public void honk() {
        System.out.println("Car honking!");
    }
}
public class Motorcycle extends Vehicle {
    public void start() {
        System.out.println("Motorcycle engine starting...");
    }
}
```

Now, write the output of the following code snippets with a detailed explanation. If there is an error, specify it with a reason.

```
1. Vehicle myVehicle = new Car();
   myVehicle.start();

2. Car myCar = new Car();
   myCar.honk();
   myCar.start();

3. Vehicle myVehicle2 = new Motorcycle();
   myVehicle2.start();
```

```
   //myVehicle2.honk(); //This line is executed if uncommented

4. Vehicle myVehicle3 = new Vehicle();
   Car myCar2 = (Car) myVehicle3;
   myCar2.honk();
```

[5] *[CSE 23, Final]*

Q28. Consider the following Java program.

(I) What would be the output if we ran the `main` method in the `B` class?

(II) What would be the output if we put the same `main` method inside `A` class and ran it from the `A` class?

```
01. public class B {
02.     private void m() { System.out.println("B main"); }
03.     public static void main(String[] args) { B b = new A(); b.m(); }
04. }
05. class A extends B {
06.     public void m() { System.out.println("A main"); }
07. }
```

[3] *[CSE 22, TT]*

Q29. Is the following code correct or incorrect? Justify your answer. If incorrect, specify why and rewrite the correct code:

```
1.  public abstract class A {
2.      final abstract void show();
3.  }
4.  public class B extends A {
5.      void show(int a) { }
6.  }
```

[5] *[CSE 17, Final; CSE 18, TT]*

Q30. Check the following Java program carefully. Do you think there is any compilation error? If there is, find the errors, correct them using the line number and then write the output. For correcting you cannot insert a new statement or delete a statement. If there is no error then write only output.

```
01.  abstract class P {     int p = 70;
02.      P(int p) { this.p = p;
03.                 System.out.println("Const: P ="+p); }
04.      void pm() { System.out.println("PM : "+p); }
05.      abstract void d();
06. }
07.  class Q extends P {    int q = 60;
08.      Q(int p, int q) { super(p); this.q = q;
09.                        System.out.println("Const: Q ="+q); }
10.      void d() { p++; System.out.println("QD : "+p); }
11. }
12.  class R extends Q {    int p = 50, r = 40;
```

```
13.      R(int p, int q, int r) { super(p,q); this.r = r;
14.                               System.out.println("Const: R ="+r);
15.      void d() { p--; System.out.println("RD : "+p); }
16. }
17.  class InheritExam {
18.      public static void main(String args[]) {
19.          R r = new R(20,20,10); r.pm(); r.d();
20.          P p = r; p.d(); p.pm(); p.p = 200; p.d(); r.d();
21.          Q q = r; q.p = 300; q.d(); r.d();
22.      }
23. }
```

[10] *[CSE 20, Final]*

Q31. Check the following Java program carefully. Do you think there is any compilation error? If there is, find the errors, correct them using the line number and then write the output. For correcting you cannot insert a new statement or delete a statement. If there is no error then write only output.

```
01.  abstract class P {  int p = 10;
02.      P(int p) { this.p = p;
03.                 System.out.println("Const: P ="+p); }
04.      void p() { System.out.println("PM : "+p); }
05.      abstract void d();
06. }
07.  class Q extends P {  int q = 20;
08.      Q(int p, int q) {
09.          super(p); this.q = q;
10.          System.out.println("Const: Q ="+q); }
11.      void d() { p++; System.out.println("Const: Q ="+p); }
12. }
13.      void d() { p++; System.out.println("QD : "+p); }
14. }
15.  class R extends Q {  int p = 30, r = 40;
16.      R(int p, int q, int r) {
17.          super(p,q); this.r = r;
18.          System.out.println("Const: R ="+r); }
19. }
20.      void d() { p--; System.out.println("RD : "+p); }
21. }
22.  class TestA {
23.      public static void main(String args[]) {
24.          R r = new R(20,20,10); r.p(); r.d();
25.          P p = r; p.d(); p.pm();
26.          p.p = 200; r.d();
27.          Q q = r; q.p = 300; q.d(); r.d();
28.      }
29. }
```

[10] *[CSE 20, TT2]*

Q32. Check the following Java program carefully. Do you think there is any compilation error?

If there is, find the errors, correct them using the line number and then write the output. For correcting you cannot insert a new statement or delete a statement. For any wrong error detection, you will get negative marks. If there is no error then write only output.

```
01.  abstract class A {     int x = 10;
02.      A(int x) { this.x = x; System.out.println("Const: A ="+x); }
03.      void m() { System.out.println("AM : "+x); }
04.      abstract void d() { System.out.println("BL A1"); }
05. }
06.  class B extends A {     int x = 20; int y = 30;
07.      { System.out.println("BL B1"); }
08.      B(int x, int y) { super(x); this.y = y;
09.          System.out.println("Const: B ="+y); }
10.      } void m() { System.out.println("BM : "+x++); }
11.      void d() { super.x++; System.out.println("BD : "+x); }
12.      static { System.out.println("SBL B2"); }
13. }
14.  class TestA { public static void main(String args[]) {
15.          B b = new B(100, 200); b.d(); b.m();
16.          A a = b; a.d(); a.m(); System.out.println(a.x+" "+b.x);
17.      }
18. }
```

[10] *[CSE 21, TT2]*

Q33. Check the following Java program carefully. Do you think there is any compilation error? If there is, find the errors, correct them using the line number and then write the output. For correcting you cannot insert a new statement or delete a statement. If there is no error then write only output.

```
01.  abstract class P {     int p = 70;
02.      P(int p) { this.p = p;
03.                 System.out.println("Const: P ="+p); }
04.      void pm() { System.out.println("PM : "+p); }
05.      abstract void d();
06. }
07.  class Q extends P {    int q = 60;
08.      Q(int p, int q) { super(p); this.q = q;
09.                        System.out.println("Const: Q ="+q); }
10.      void d() { p++; System.out.println("QD : "+p); }
11. }
12.  class R extends P {     int p = 50;
13.      R(int p) { super(p); }
14.      void d() { p--; System.out.println("Const: R ="+p); }
15. }
16.  class InheritExam {
17.      public static void main(String args[]) {
18.          Q q = new Q(11,22); q.p = 300;     q.d();
19.          Q.R r = q.new R(30,10); r.pm(); r.d();
20.          P p = r; p.d(); p.pm(); p.p = 200;    r.d();
21.      }
22. }
```

[10] *[CSE 21, Final]*

Q34. Check the following Java program carefully. Is there any compilation error? If there are, find the errors, correct them using the line number, and then write the output. You cannot insert a new statement or delete a statement to correct. If there is no error, then write only the output.

```
01.  abstract class A {     int a = 55;
02.      A(int a) {
03.          System.out.println("Const: A ="+this.a);
04.          this.a = a; }
05.      }
06.      void am() { System.out.println("AM : "+a); }
07.      abstract void d();
08. }
09.  class B extends A {     int b = 45;
10.      B(int a, int b) { super(a); this.b = b;
11.          System.out.println("Const: B ="+(a+b)+"+"+a); }
12. }
13.      void d() { a++; System.out.println("BD : "+a+" "+b); }
14.      class C extends A {     int a = 35, c = 25;     b = -10;
15.          C(int a, int c) { super(a); this.a = c;
16.              System.out.println("Const: C = "+b+"+"+c);
17.          }
18.          void d() { a--; System.out.println("CD : "+a+" "+c); }
19. }
20.  class InheritExam {
21.      public static void main(String args[]) {
22.          B q = new B(44,33); q.a = 99;     q.d();
23.          B.C r = q.new C(12,11); r.am(); r.d();
24.          A a = r; a.d(); a.am() = 200; r.d();
25.      }
26. }
```

[10] *[CSE 22, Final]*

Q35. Check the following Java program carefully. Is there any compilation error? If so, identify the error using the line number and provide the correction. Finally, write the output of the program.
Note: You cannot insert or delete statements to fix errors.

```
01.  abstract class A { int a = -11;
02.      A(int a) { this.a = a; 10.println("C: A = "+a); }
03.      void am() { 10.println("AM : "+a); }
04.      abstract void d();
05. }
06.  class B extends A { int a = 22, b = 33;
07.      B(int a, int b) { super(a); this.a = b;
08.          10.println("C: B = "+(a+b)+"+a); }
09.      void d() { a++; 10.println("BD : "+a+" "+b); }
10. }
```

```
11.  class C extends B { int c = 0;
12.      C(int a, int c) { super(a, a); this.c = c;
13.          10.println("C: C = "+b+"+"+c); }
14.      void d() { a--; 10.println("CD : "+a+" "+c); }
15. }
16.  class InheritExam {
17.      public static void main(String args[]) {
18.          B q = new B(55,66); q.a = 99; q.d();
19.          B.C r = q.new C(77,88); r.am(); r.d();
20.          A a = r; a.d(); a.am = 100; r.d();
21.      }
22. }
```

[10] *[CSE 23, Final]*

Q36. Write a Java GUI program in which you are required to extend JFrame. Create three text fields to take input of height, width, depth for a Box object. Clicking on 'Create' button will create a new Box object and add it into an ArrayList. Clicking on 'View' button will print the total ArrayList into console output using for-each version of for loop. Add necessary labels for the input textfields. (Note: integrates Box class design + Swing.)**[10]** *[CSE 16-17, Final]*

Q37. Given an ArrayList with duplicate values, the task is to remove the duplicate values from this ArrayList in Java.
Input: List = {1, 10, 2, 2, 10, 3, 3, 3, 4, 5, 5}
Output: List = {1, 10, 2, 3, 4, 5} **[10]** *[2017-18 Akash]*

Q38. Consider the class `Q2` provided below. Your task is to implement class `Y` such that running the `main` method produces the specified output.
Restriction: Class `Y` must not contain any print statements.

```
01. abstract class X { abstract int m(int i); }
02. public class Q2 {
03.     public static void main(String[] args) {
04.         Y y = new Y(); IO.println(y.m(5));
05.         X x = y; IO.println(x.m(5));
06.         Y.Z z = y.new Z(3); IO.println(z.m(4));
07.         IO.println(Y.m2("hello"));
08.     }
09. }
```

Expected Output:

```
15
15
7
HELLO
```

(Source corrected: this question appears in CSE 23, TT2.)

[6] *[CSE 22, TT2]*

Q39. Consider the following class `Test` and interface `B`. You cannot change this class or interface.

```
01. interface B { int m(int i); }
02. public class Test {
03.     public static void main(String[] args) {
04.         System.out.print(A.a); A a = new A(20);
05.         System.out.println(" "+a); System.out.print(a.a);
06.         System.out.print(" "+a.m(30)); System.out.print(" "+a.m(15));
07.         B b = a; System.out.println(" "+b.m(40));
08.         A.C c = a.new C(); System.out.println(c.m("Welcome"));
09.         System.out.println(c.m("Java"));
10.     }
11. }
```

Now write a Java Class named `"A"`, so that if we run the `main` method in the `Test` class it gives the following output:

```
10 A class
20 50 35 60
Welcome C class
Java C class
```

[7] *[CSE 22, TT]*

Q40. Consider the following interface `Action`, and classes `MainApp`, `Fish`. You cannot change these classes or interfaces.

```
01.  interface Action { public void move(); }
02.  class Fish extends Animal {
03.      public Fish(double weight, String movementType)
04.      { super(weight, movementType); }
05.      public void printPrice(double pricePerKg)
06.      { System.out.println(getWeight()*pricePerKg); }
07. }
08.  class MainApp {
09.      public static void main(String[] args) {
10.          Fish shark = new Fish(350.0, "Swimming"); shark.eat("Tuna");
11.          shark.sleep(120); shark.printPrice(200);
12.          Action action = shark; action.move();
13.          Animal lion = new Fish(250.0, "Running"); lion.eat("Deer");
14.          lion.sleep(240); action = lion; action.move();
15.      }
16. }
```

Now write a Java Class named "`Animal`" which will have only private instance variables, so that if we run the `MainApp` class it gives the following output:

```
Eating Tuna...
Sleeping for 120 min...
70000.0
Swimming
Eating Deer...
Sleeping for 240 min...
Running
```

[10] *[CSE 20, Final]*

Q41. Consider the following interface `Action`, and classes `MainApp`, `Fish`. You cannot change these classes or interfaces.

```
01.  interface Action { public void move(); }
02.  class Fish extends Animal {
03.      public Fish(double weight, String movementType)
04.      { super(weight, movementType); }
05.      public void printPrice(double pricePerKg)
06.      { return getWeight()*pricePerKg; }
07. }
08.  class MainApp { static int lim = 0;
09.      public static void main(String[] args) {
10.          static void pSleep(msg) {System.out.println(lim+"+":"+msg);}
11.          Fish shark = new Fish(350.0, "swimming"); pShark.eat("Tuna");
12.          pShark.sleep(120); pShark.printPrice(200);
13.          Action action = shark; action.move();
14.          Animal lion = new Fish(150.0, "Running"); pLion.eat("Deer");
15.          pLion.sleep(240); action = lion; action.move();
16.      }
17. }
```

Now write a Java Class named "Animal" which will have only private instance variables, so that if we run the `MainApp` class it gives the following output:

```
0: Eating Tuna...
1: Sleeping for 120 min...
2: 70000.0
3: Swimming
4: Eating Deer...
5: Sleeping for 240 min...
6: Running
```

[10] *[CSE 21, Final]*

Q42. Consider the following interface `Action`, and classes `MainApp`, `Fish`. You cannot change these classes or interfaces.

```
01.  interface Action { String move(); }
02.  class Fish extends Animal {
03.      public Fish(double weight, String movementType)
04.      { super(weight, movementType); }
05.      public double getPrice(double pricePerKg)
06.      { return getWeight()*pricePerKg; }
07. }
08.  class MainApp { static int line = 0;
09.      static void p(String msg) {System.out.println(line++ +": "+ msg);}
10.      public static void main(String[] args) {
11.          Fish shark = new Fish(350.0, "Swimming"); p(shark.eat("Tuna"));
12.          p(shark.sleep(120)); p("Shark's price: "+shark.getPrice(200));
13.          Action action = shark; p(action.move());
```

```
14.          var lion = new Animal(250.0, "Running"); p(lion.eat("Deer"));
15.          p(lion.sleep(240)); p("Lion's weight: "+lion.getWeight());
16.          action = lion; p(action.move());
17.      }
18. }
```

Now write a Java Class named "**Animal**" which will have only private instance variables, so that if we run the `MainApp` class it gives the following output:

```
0: Eating Tuna...
1: Sleeping for 120 min...
2: Shark's price: 70000.0
3: Swimming
4: Eating Deer...
5: Sleeping for 240 min...
6: Lion's weight: 250.0
7: Running
```

[10] *[CSE 22, Final]*

Q43. Analyze the `MissionControl` class and its usage of the `Navigable` interface. Implement the `Spacecraft` class with only private instance variables to produce the target output.

```
01.  interface Navigable {String move();}
02.  class Satellite extends Spacecraft {
03.      Satellite(double fuel, String orbitType) {
04.          super(fuel, orbitType);
05.      }
06.      public double getDistance(double speed) {
07.          return getFuelLevel() * speed;
08.      }
09. }
10.  public class MissionControl { static int line = 0;
11.      static void log(String msg) {10.println(line++ + ": " + msg);}
12.      public static void main(String[] args) {
13.          Satellite voyager = new Satellite(500.0, "Interstellar");
14.          log(voyager.scan("Jupiter")); log(voyager.signal(50));
15.          log("Distance: " + voyager.getDistance(150));
16.          Navigable nav = voyager; log(nav.move());
17.          var rover = new Spacecraft(120.0, "Planetary");
18.          log(rover.scan("Mars Crater")); log(rover.signal(15));
19.          log("Fuel remaining: " + rover.getFuelLevel());
20.          nav = rover; log(nav.move());
21.      }
22. }
```

Target Output:

```
0: Scanning Jupiter...
1: Signal strength: 50%
2: Distance: 75250.0
3: Interstellar
```

```
4: Scanning Mars Crater...
5: Signal strength: 15%
6: Fuel remaining: 120.0
7: Planetary
```

Constraints: (1) You cannot modify class `MissionControl`, `Satellite`, or interface `Navigable`. (2) All instance variables in `Spacecraft` must be private.

[10] *[CSE 23, Final]*

7. Packages & Interfaces

Q1. Why is it possible for an interface to extend another two interfaces in java? **[2]** *[2017-18 Akash]*

Q2. What happens if a class partially implements an interface? **[2]** *[2017-18 Akash; CSE 16-17, Final]*

Q3. Differentiate between "`protected`" and "`default`" access specifiers. *[CSE 16-17, Final; CSE 17, Final; CSE 18, TT]* **[3]** *[CSE 16-17, Final]*

Q4. Suppose packages A and B both have classes named C. Now, what will happen if both the packages are imported using the * form? **[2]** *[CSE 19, Final]*

Q5. Can you import same package or class twice? Will the JVM load the package twice at runtime? How many objects will be created in the following code?

```
String s1="Welcome";
String s2="Welcome";
String s3="Welcome";
```

[3] *[CSE 16-17, Final]*

Q6. Define True/False. If false give the correct answer.

i. Inner class cannot be public.
ii. Since constructor does not have any return type, it cannot be overloaded.
iii. Protected members of a class cannot be accessed from non-subclass within same package.
iv. Abstract class cannot have static method.
v. Static interface methods are not inherited by either an implementing class or a sub-interface.

[5] *[2017-18 Akash]*

Q7. Define True/False. If false give correct answer.

i. The default layout manager of Java Swing frame is flowlayout.
ii. Default (access modifier) members of a class can only be accessible within different package.
iii. Unlike *Hashtable*, *HashMap* is synchronized.
iv. *this()* can be called from any non-static method in a class.
v. Super class object can be referenced by subclass variable.

[5] *[CSE 16-17, Final]*

Q8. Define True/False. If false give correct answer.

i. Java does not support multiple inheritance for Interfaces.
ii. All methods in an Interface must be abstract and public.
iii. *RuntimeException* is a checked exception.
iv. An abstract classdon't have any constructor.
v. Interfaces may have abstract constructor.

[5] *[CSE 16-17, Final]*

Q9. What will be the output of the following code? Explain.

```
package pkgA;                          | public class clover{
public class frog{                     |   public static void main(String[] args){
  int a = 5; protected int b = 6;|     |     Frog f = new Frog();
  public int c = 7;                    |     System.out.println("a = " + f.a);
}                                      |     System.out.println("b = " + f.b);
package pkgB;                          |     System.out.println("c = " + f.c);
import pkgA.*;                         |   }
                                       | }
```

[5] *[2017-18 Akash]*

Q10. Write True/False in the given box.

i. Java supports the implementation of multiple interfaces in a single class.
ii. Methods that are declared as final cannot be overridden.
iii. An abstract class can inherit an interface.
iv. In Java, class names must start with an uppercase letter.
v. Wrapper classes allow primitive types to be accessed as objects.

[5] *[CSE 17, Final; CSE 18, TT]*

Q11. What are the 4 different access specifiers in Java? In which cases do public, protected, and default work differently from each other? **[1+2]** **[3]** *[CSE 19, TT2]*

Q12. Summarize in a table how the class members are accessed under the same or different class or subclass or package. **[5]** *[CSE 20, Final; CSE 21, Final]*

Q13. Check the following Java program carefully. Do you think there is any compilation error? If there is, find the errors, correct them using the line number and then write the output. For correcting you cannot insert a new statement or delete a statement. If there is no error then write only output.

```
01.  interface A {       int a = 10; void am(); }
02.  interface B extends A {
03.      static int b = 30; int c = 50;
04.      interface C {  final static int a = 40;
05.          void bm(); void am();
06.      }
07.      abstract void bm();
08. }
09.  class TestB implements B, C { int x = 100; private int p = 0;
```

```
10.      public static void main(String args[]) {
11.          TestB t = new TestB();
12.          t.s();              t.am();
13.          A a = t; B b = t; C c = t;
14.          a.am();         b.am();     b.bm();     c.bm();
15.      }
16.      public class D {
17.          D() { int y = 40; x = 90;
18.              System.out.println("x is: " + x + " and y is: " + y);
19.              public void s() {
20.                  System.out.println("x is: " + x + " p = " + p);
21.              }
22.          }
23.          public void am() { System.out.println("AM: "+c); }
24.          public void bm() { System.out.println("BM: "+a); }
25.          void s() { x = 10;      y = 20;
26.              System.out.println("x is: " + x + " and y is: " + y);
27.              D d = new D();      d.s();
28.          }
29.      }
30. }
```

[10] *[CSE 20, TT2]*

Q14. Check the following Java program carefully. Do you think there is any compilation error? If there is find the errors, correct them using the line number and then write the output. For correcting you cannot insert a new statement or delete a statement. If there is no error then write only output.

```
01.  interface A {       int a = 80;  void am(); }
02.  interface B extends A { static int b = 10;    int c = 20;
03.       interface C {        final static int a = 50;
04.            void bm();          void am();
05.       }
06.  abstract void bm();
07. }
08.  class InterfaceExam implements B,C {int x = 100; private int p = 0;
09.      public static void main(String args[]) {
10.          InterfaceExam t = new InterfaceExam();
11.          f.s(); t.am(); A a = t; B b = t;     C c = t;
12.          a.am();    b.am();         b.bm();       c.bm();
13.      }
14.      public class D {
15.          D() { int y = 40; x = 90;
16.              System.out.println("x is: " + x + " and y is: " + y); }
17.          public void s() {
18.              System.out.println("x is: " + x + " p = " + p); }
19.      }
20.      public void am() { System.out.println("AM: "+c); }
21.      public void bm() { System.out.println("BM: "+a); }
22.      void s() { x = 60;      int y = 70;
```

```
23.          System.out.println("x is: " + x + " and y is: " + y);
24.          D d = new D();      d.s();
25.      }
26. }
```

[10] *[CSE 20, Final]*

Q15. Consider the following class `TestB`, that uses interfaces A, B, and C. You cannot change the `TestB` class.

```
01.  class TestB implements B, B.C {  int a = 20;
02.      public static void main(String args[]) {
03.          TestB t = new TestB(); t.am(1); t.bm(); t.cm();
04.          B b = t; B.C c = t; A a = b; A.bm();
05.          a.am(2); b.am(3); b.bm(); c.bm(); c.cm();
06.          System.out.println(A.a+" "+B.a+" "+B.b+" "+B.c+" "+C.a);
07.      }
08.      public void am(int i) { System.out.println("AM: = "+c); }
09.      public void bm() { System.out.println("BM: = "+a); }
10. }
```

Now write Java Interfaces named "A", "B", and "C", so that if we run the `TestB` class it gives the following output:

```
AM: = 50
BM: = 20
C.cm()
A.bm()
AM: = 50
AM: = 50
BM: = 20
BM: = 20
C.cm()
10 10 30 50 40
```

[10] *[CSE 21, TT2]*

Q16. Consider the following Java program `TestB`, that uses interfaces A, B, and C. You cannot change the `TestB` class.

```
01.  class InterfaceExam implements B, C { int a = 20;
02.      public static void main(String args[]) {
03.          System.out.println("Start");
04.          InterfaceExam t = new InterfaceExam(); t.am(); t.bm();
05.          System.out.println("c = " + t.cm()); B b = t; C c = t;
06.          A a = b; System.out.println("a = " + A.bm());
07.          a.cm(); b.bm();        b.cm();   c.bm(); c.cm();
08.          System.out.println("a = " + a); a.bm(); c.bm();
09.          System.out.println(a.cm()+"-B.a="+B.a+"-C.a="+C.a);
10.      }
11.      public void am(int i) { System.out.println("AM: "+i); }
12.      public void bm() { System.out.println("BM: "+a); }
13. }
```

Now write Java interfaces named "A", "B", and "C", so that if we run the main method in `InterfaceExam` class it gives the following output:

```
Start
5.AM: = 55
6.BM: = 20
6.cm() = 240
3.AM:bm()
5.AM: = 55
6.BM: = 20
6.BM: = 20
a.cm()
public void am(int i) { System.out.println("AM: "+i); }
public void bm() { System.out.println("BM: "+a); }
Start
5.AM: = 55
6.BM: = 20
5.AM: = 55
6.BM: = 20
6.BM: = 20
6.BM: = 20
11 11 33 55 44
```

[10] *[CSE 21, Final]*

Q17. Consider the following Java program `TestB`, that uses interfaces A, B, and C. You cannot change the `TestB` class.

```
01.  class InterfaceExam implements B, C {     int a = 50;
02.      static int line = 0;
03.      static void p(String msg) {System.out.println(line++ +": "+ msg);}
04.      public static void main(String args[]) { p("Start");
05.          InterfaceExam t = new InterfaceExam(); t.am(); t.bm();
06.          p(t.cm()); B b = t; C c = t;
07.          A a = b; p(A.bm());
08.          a.cm();     b.am();        b.bm(); c.bm(); c.cm();
09.          p(c.cm());
10.          p(A.a+"-"+B.a+"-"+B.b+"-"+B.c+"-"+C.a);
11.      }
12.      public void am(int i) { System.out.println("AM: "+i); }
13.      public void bm() { System.out.println("BM: "+a); }
14. }
```

Now write Java interfaces named "A", "B", and "C", so that if we run the **main** method in **InterfaceExam** class it gives the following output:

```
0: Start
1: AM: = 30
2: BM: = 50
3: C.cm: = 40
4: A.bm: = 10
5: AM: = 30
```

```
6: AM: = 30
7: BM: = 50
8: BM: = 50
9: C.cm: = 40
10: 10 10 20 30 40
```

[10] *[CSE 22, Final]*

Q18. Consider the following class `InterfaceExam`, that uses interfaces A, B, and C. Provide the implementation of the Java Interfaces named "A", "B", and "C" so that the program generates the exact output shown on the right.

```
01.  public class InterfaceExam implements B, C {
02.      int a = 70; static int line = 0;
03.      static void p(String msg) {
04.          10.println(line++ + ": " + msg);
05.      }
06.      public static void main(String[] args) {
07.          p("Begin");
08.          InterfaceExam t = new InterfaceExam();
09.          B b = t; C c = b; b.bm(); c.am(); b.am(t);
10.          p(b.cm()); p(c.cm());
11.          A a = (A) am(t); t.cm(t); t.bm(t.am(t)); t.bm();
12.          b = (B) b; t.am(t); B.b = 25; t.bm();
13.          c.bm(); p(A.a + "-" + B.b + "-" + C.a);
14.      }
15.      public void am(int i) { p("AM "+i); }
16.      public void am(double a) { p("AM "+a); }
17.      public void am(int a) { p("Class AM"); }
18.      public String cm() { return "Class CM"; }
19. }
```

Target Output:

```
0: Begin
1: Class BM
2: Class IM
3: Class CM
4: AM-5
5: AM-5.0
6: A.bm
7: AM-1
8: AM-10
9: Class BM
10: 10 20 99
```

Constraints: You cannot change the `InterfaceExam` class.

[10] *[CSE 23, Final]*

8. Exception Handling

Q1. What is the difference between exception and error in JAVA? **[2]** *[2017-18 Akash]*

Q2. Differentiate `throw` and `throws`. **[2]** *[CSE 19, Final]*

Q3.
```
try{ ........ }
catch(arrayIndexOutOfBoundsException ex1){ ...... }
catch (Exception ex2){ ...... }
catch (ArithmeticException ex3){ ...... }
```
What type of issue may arise in this try-catch block? Why?

[2] *[CSE 19, Final]*

Q4. Are `NullPointerException` and `IllegalAccessException` (CSE 20) / `ArrayIndexOutOfBoundsException` and `ClassNotFoundException` (CSE 21) checked or unchecked exceptions? **[5]** *[CSE 20, Final; CSE 21, Final]*

Q5. Differentiate between Checked Exception and Unchecked Exception? **[3]** *[CSE 16-17, Final]*

Q6. Write on throws keyword in Java? How can you create a new custom unchecked Exception class. **[3+2]** **[5]** *[CSE 16-17, Final]*

Q7. Is there any way to throw an exception from a method without adding throws declaration? If possible show an example, if not write the reason. **[5]** *[2017-18 Akash]*

Q8. Show an example of "Array index out of bound exception" by writing a Java Program. **[5]** *[2017-18 Akash]*

Q9. Write a method maintaining the given criteria.

i. Suppose the method is given an array of integers numbers. This array will be traversed and each number will be printed. Inverted numbers should be printed in the method.
ii. If any number is found zero the program will throw an `ArithmeticException` and print that "A Zero Is Found!!!".
iii. Whether any exception occurs or not the method will end printing "Array Traversing Stopped!!!".

[5] *[CSE 19, Final]*

Q10. What is the difference between error and **exception**? What is the purpose of finally? To create an exception which class do you have to extend? In which case **throws** keyword should be used? **[1+2+1+1]** **[5]** *[CSE 19, TT2]*

Q11. Write the output of the following program with explanation.

```
public class TestException {
  void test() {
    try {
      String s = null;
      try {
        System.out.println(s.length());
        try {
          System.out.println("inner");
          throw e;
        } catch (NullPointerException e) {
```

```
            System.out.println("1");
          }
          System.out.println("2");
        } catch (NullPointerException e) {
          System.out.println("outer");
          return;
        } finally {
          System.out.println("3");
        }
        System.out.println("4");
      }
    }
    public static void main(String args[]) {
      TestException ob = new TestException();
      ob.test();
    }
}
```

[10] *[CSE 16-17, Final]*

Q12. Write down the output of the following program:

```
static void procA(){
    try{
        System.out.println("inside procA");
        return;
    }finally{
        System.out.println("ProcA's finally");
    }
}
static void procB(){
    try{
        System.out.println("inside procB");
        throw new RuntimeException("Demo");
    }finally{
        System.out.println("ProcB's finally");
    }
}
public static void main(String[] args){
    procA();
    try{
        procB();
    }catch(Exception e){
        System.out.println("Exception handled");
    }
}
```

[5] *[CSE 19, Final; CSE 23, Final]*

Q13. Check the following java program carefully.

```
01 class ExceptionExam {
```

```
02   public static void main() throws NullPointerException("Null") {
03     public static void main() {
04       try { System.out.println(a[5] + "=eh"); return; }
05       finally { System.out.println("noException"); }
06     public static void main(String[] args) {
07       int a = new int[10];
08       System.out.println("Total Exceptions: "+a);
09       if (a == 0) a = 10; m = a;
10       try {
11         if (a == 1) a = a / (a - a);
12         if (a > 2) { int a[] = {1}; a[1] = 19; }
13                     System.out.println("a[1] = " + a[1]);
14         else m++;
15       } catch (ArrayIndexOutOfBoundsException ae) {
16         System.out.println("Throw: m, a");
17         finally { System.out.println("ArrayIndexOutOfBoundsException: "+ae);
18         } catch (Exception e) {
19           System.out.println("Exception"+e);
20         } finally { System.out.println("Exam Completed"); }
21         System.out.println("Show Index");
22       }
23     }
24   }
```

and write the output for the following commands:

i. `java ExceptionExam`
ii. `java ExceptionExam a1`
iii. `java ExceptionExam a1 a2`
iv. `java ExceptionExam a1 a2 a3`
v. `java ExceptionExam a1 a2 a3 a4`

[10] *[2017-18 Akash]*

Q14. Check the following Java program carefully,

```
01.  class ExceptionExam {
02.      public static void main() {
03.          throw new NullPointerException("Null");
04.      }
05.      public static void main(String[] args) {
06.          try { System.out.println("e = "+ e); return;
07.          } finally { System.out.println("not-reachable"); }
08.      }
09.      public static void main(String[] args) { int exc = exc.length;
10.          System.out.println("Total Exceptions: " + e);
11.          try{ if (e == 1) e = e / (e - e);
12.              try{ if (e == 3) e = e / (e - 3); int a[] = { 1, 2, 3 };
13.                  a[5] = 99; System.out.println("a[1] = " + a[1]);
14.                  if (e == 3) e = e;
15.                  else e=e;
16.              }catch (ArrayIndexOutOfBoundsException e) {
17.                  System.out.println("Divide by 0"); }
```

```
18.          }finally { System.out.println("Un-reachable code"); }
19.          }catch (ArithmeticException) NullPointerException e) {
20.              System.out.println("Array index out-of-bounds or Null");}
21.          }catch (Exception e) { System.out.println("Any Exception"); }
22.          finally { System.out.println("Program Completed"); }
23.          System.out.println("Exam Ended");
24.      }
25. }
```

and write the output for the following commands:

```
i)   java ExceptionExam
ii)  java ExceptionExam e1
iii) java ExceptionExam e1 e2
iv)  java ExceptionExam e1 e2 e3
v)   java ExceptionExam e1 e2 e3 e4
```

[10] *[CSE 20, Final]*

Q15. Check the following Java program carefully,

```
01.  class ExceptionExam {
02.      public static void main() {
03.          class A extends NullPointerException
04.          { A() {super("NewExc"); } }
05.          throw new A();
06.      }
07.      public static void main(int e) {
08.          try { System.out.println("e = " + e); return;
09.          } finally { System.out.println("not-reachable"); }
10.      }
11.      public static void main(String exc[]) {
12.          System.out.println("Total Exceptions: " + e);
13.          if(e == 0) e = 10 / e;
14.          try { if (e == 2) int a[] = { 1, 2, 3 }; a[5] = 99;
15.              if (e == 3) int a[] = { 1, 2, 3 }; a[5] = 99;
16.              else if (e == 3) e = e/(e-3);
17.          } catch (ArrayIndexOutOfBoundsException e) {
18.              System.out.println("Divide by 0"); }
19.          } catch (ArithmeticException e) {
20.              System.out.println("Array index out-of-bounds"+e);
21.          } catch (Exception e) { System.out.println("Any Exception"); }
22.          finally { System.out.println("Program Completed"); }
23.          System.out.println("Exam Ended");
24.      }
25. }
```

and write the output for the following commands:

```
i)   java ExceptionExam
ii)  java ExceptionExam e1
iii) java ExceptionExam e1 e2
```

```
iv)  java ExceptionExam e1 e2 e3
v)   java ExceptionExam e1 e2 e3 e4
```

[10] *[CSE 21, Final]*

Q16. Check the following Java program carefully.

```
01.  class E extends ClassNotFoundException
02.      { E() {super("Class Not Found"); } }
03.  public class ExceptionExam {
04.      public static void m() throws E { throw new E(); }
05.      public static void m(int e) {
06.          try { System.out.println("e = " + e); return;
07.          } finally { System.out.println("Not Executed!"); }
08.      }
09.      public static void main(String exc[]) { int e = exc.length;
10.          System.out.println("Total Arguments: " + e);
11.          if (e == 0) e = 100 / e;
12.          if (e == 1) e = e / (e - e);
13.          try { if (e == 2) {int a[] = { 1, 2, 3 }; a[5] = 99;
14.              if (e == 3) m();
15.          } catch (ArrayIndexOutOfBoundsException e) {
16.              System.out.println("Class Found"); }
17.          } finally { System.out.println("Un-reachable code"); }
18.          } catch (ArithmeticException e) {
19.              System.out.println("Array index not found");
20.          } catch (Exception e) { System.out.println("Exception:"+e);
21.          } finally { System.out.println("Finally Done"); }
22.          System.out.println("Exception Ended");
23.      }
24. }
```

Now write the output for the following commands:

```
i)   java ExceptionExam
ii)  java ExceptionExam E1
iii) java ExceptionExam E1 E2
iv)  java ExceptionExam E1 E2 E3
v)   java ExceptionExam E1 E2 E3 E4
```

[10] *[CSE 22, Final]*

Q17. Check the following Java program carefully.

```
01.  class E extends InterruptedException
02.      { E() {super("Execution Interrupted");}}
03.  public class ExceptionExam {
04.      public static void m() throws E { throw new E(); }
05.      public static void m(int e) { 10.println("e = " + e);
06.          try { if (e > 3) System.exit(0); return;
07.          } finally { 10.println("Not Executed!"); }
08.      }
```

```
09.      public static void main(String exc[]) { int e = exc.length;
10.          try { 10.println("e = " + e);
11.              if (e > 5) { String a = null; a.length(); }
12.              if (e == 2) { 10.println("e =" + e); m(e); }
13.              { catch (StringIndexOutOfBoundsException e) {
14.                  10.println("Index Out Of Bounds"); }
15.              } finally { 10.println("Un-reachable code!"); }
16.          } catch (ArithmeticException e) { 10.println("Zero!");
17.          } catch (NullPointerException e) { 10.println("Null!");
18.          } catch (Exception e) { 10.println("Exception:"+e);
19.          } finally { 10.println("Finally Done"); }
20.          10.println("Exception Ended");
21.      }
22. }
```

Now write the output for the following commands:

```
i.   java ExceptionExam
ii.  java ExceptionExam E1
iii. java ExceptionExam E1 E2
iv.  java ExceptionExam E1 E2 E3
v.   java ExceptionExam E1 E2 E3 E4
```

[10] *[CSE 23, Final]*

9. Multithreaded Programming

Q1. What are the ways in which you can instantiate a thread? What are the advantages of using multiple threads in a program? **[5]** *[2017-18 Akash]*

Q2. Write a code using Java threading where a Deadlock condition must occur. **[5]** *[2017-18 Akash]*

Q3. How can you make your java method thread safe? What is the purpose of `join()` method in Thread class? **[2+3]** **[5]** *[CSE 16-17, Final]*

Q4. What is the key advantage of using multithreading? How can multiple threads be created in Java? How is synchronization accomplished in a multithreading environment? **[5+5]**[5] *[CSE 19, Final]*

Q5. How is synchronization accomplished in a multi-threading environment?**[5]** *[CSE 20, Final]*

Q6. Draw a diagram showing all the `Thread` states and how they relate to each other.**[5]** *[CSE 20, Final; CSE 21, Final]*

Q7. Explain deadlock. Why is inter-thread communication necessary? What are the different possible states of a thread. **[5]** *[CSE 19, Final]*

Q8. `wait()`, `notify()` and `notifyAll()`– these three methods are available in every class. Why? **[5]** *[CSE 19, Final]*

Q9. Write the output of the following program. Explain why the output appears in the order it does.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running.");
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
        }
        System.out.println("Thread finished.");
    }
}
public class ThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("Main starting.");
        MyThread t = new MyThread();
        t.start();
        System.out.println("Main after start.");
        t.join();
        System.out.println("Main finishing.");
    }
}
```

[5] *[CSE 23, Final]*

Q10. Once there was a god keeper in Portugal who used to believe in luck. He throws dice to decide whether he should jump left or right during penalty shootout. If he used to get three or five he used to jump left. Penalty shooters also had two options like the goalkeeper. If both the goal keeper and shooter chose the same side then the goal is saved. If they chose opposite sides then it was celebrated as goal. Now, implement this goalkeeper-penalty shooter problem for five penalties by using multi-threaded programming. **[10]** *[2017-18 Akash]*

Q11. Consider the following class `Producer`, `Consumer`, and `Q`. Change the `Q` class so that `Producer` and `Consumer` communicate with each other, so that, `Consumer` waits until `Producer` puts a new value and `Producer` waits until `Consumer` gets the already put value.

```
01.  class Q { int n;
02.      synchronized int get() { return n; }
03.      synchronized void put(int n) { this.n = n; }
04. }
05.  class Producer implements Runnable {
06.      Q q; Thread t;
07.      Producer(Q q) { this.q = q;
08.          t = new Thread(this, "Producer"); }
09.      public void run() { int i = 0;
10.          while(true) { q.put(i++); }
11.      }
12. }
```

```
13.  class Consumer implements Runnable {
14.      Q q; Thread t;
15.      Consumer(Q q) { this.q = q;
16.          t = new Thread(this, "Consumer"); }
17.      public void run() { while(true) { q.get(); } }
18. }
```

[5] *[CSE 20, Final; CSE 21, Final]*

Q12. Write down code for two class named `T1` and `T2` where both creates a separate thread that prints "Happy Threading" but each character is printed with 0.5 second delay. `T1` extends the `Thread` class and uses priority 7 but `T2` implements `Runnable` and uses priority 3. Both classes should contain the main method that creates the object of that class and run the thread code. Also, the `main` thread should end last after finishing all other threads. **[10]** *[CSE 20, Final]*

10. Enumerations, I/O, Generics

Q1. Write down the two restrictions that apply to enumerations. *[CSE 20, Final; CSE 21, Final]***[2]** *[CSE 20, Final]*

Q2. What is the purpose of using enumeration in Java? **[2]** *[CSE 21, Final]*

Q3. What is Type Wrapper in Java? **[2]** *[CSE 19, Final]*

Q4. Define boxing and unboxing with examples. **[2]** *[CSE 19, Final]*

Q5. Write statements to show autoboxing and autounboxing. **[2]** *[CSE 19, Final]*

Q6. What is Generics? What flexibility it provides while programming in Java. **[2]** *[CSE 19, Final]*

Q7. What are the restrictions of generics? How generics improve type safety? Write a generic method reverseArray that reverses the order of elements in an array. Use this generic method in a program to reverse arrays of different types. Print each array both before and after calling reverseArray method. **[2+2+6]** **[6]** *[2017-18 Akash]*

Q8. What are the advantages of using generics over simply type-casting? Show How type bounding is useful in using generics? **[5]** *[CSE 21, Final]*

Q9. What is `Erasure`? Explain the four restrictions when using generics. *[CSE 21, Final; CSE 20, Final]* **[5]** *[CSE 21, Final]*

Q10. What is `Erasure`? When type erasure requires a bridge method? Explain with an example. **[5]** *[CSE 20, Final]*

Q11. What is *Wildcard argument*? The following program has an error; explain the reason behind that error. How can we fix that error?

```
01.  class A<T extends Number> { T n; A(T t) { n = t; }
```

```
02.      boolean isSame(A<?> ob) { if(n == ob.n) return true;
03.          return false; }
04. }
05.  public class GenericsExam {
06.      public static void main(String[] args) {
07.          Integer i = 5; Double d = 1.1;
08.          A<Integer> iob = new A<Integer>(i);
09.          A<Double> dob = new A<Double>(d);
10.          if(iob.isSame(dob)) System.out.println("Same.");
11.          else System.out.println("Differ.");
12.      }
13. }
```

[5] *[CSE 21, Final]*

Q12. Write a Generic method `minimum` that finds the minimum for the following three cases.

```
public static void main(String args[]){
    System.out.println("Min: " + minimum (-3, 40, 15));
    System.out.println("Min: " + minimum (16.6, 8.8, 7.7));
    System.out.println("Min: " + minimum ("mango", "guava",
                                          "orange"));
}
```

[10] *[CSE 19, Final]*

Q13. Where is the location of `RetentionPolicy` in Java package? What are the policies available in Java? **[2]** *[CSE 20, Final; CSE 21, Final]*

Q14. Consider the following class `EnumExam`, and the use of `Mango`.

```
01.  public class EnumExam {
02.      public static void main(String[] argv) {
03.          for(Mango m : Mango.values())
04.              System.out.println("Position: "+m.ordinal()
05.              +", Name: "+m+", Price: "+m.getPrice()+" Tk.");
06.      }
07. }
```

Now write a Enumeration named "Mango" in such way so that the output of this program should look like this.

```
Setting default price 50Tk.
Setting default price 50Tk.
Setting default price 50Tk.
Position: 0, Name: Failer, Price: 150 Tk.
Position: 1, Name: Langda, Price: 120 Tk.
Position: 2, Name: Gopalbogh, Price: 50 Tk.
Position: 3, Name: Himsagor, Price: 70 Tk.
Position: 4, Name: Khirsapat, Price: 50 Tk.
Position: 5, Name: Mohanbhog, Price: 50 Tk.
```

[5] *[CSE 20, Final]*

Q15. Consider the following class `EnumExam`, and the use of `E`.

```
01.  public class EnumExam {
02.      public static void main(String[] argv) {
03.          for(E e : E.values()) System.out.println("Position: "+e.ordinal()
04.          +", Name: "+e+", Value: "+e.m1()+" Value2:"+e.m2());
05.      }
06. }
```

Now write a Enumeration named "E" in such way so that the output of this program should look like this:

```
Position: 0, Name: A, Value: 50 Value2: AAA
Position: 1, Name: B, Value: 10 Value2: BBB
Position: 2, Name: C, Value: 70 Value2: CCC
Position: 3, Name: D, Value: 70 Value2: KKK
Position: 4, Name: E, Value: 30 Value2: MMM
Position: 5, Name: X, Value: 20 Value2: XXX
Position: 6, Name: Y, Value: 5  Value2: FFF
```

[5] *[CSE 21, Final]*

Q16. Consider the following Java program with class `AutoboxExam` that shows autoboxing and auto-unboxing taking place and mark them.

```
01.  public class AutoboxExam {
02.      static int m(Integer v) { return v; }
03.      public static void main(String[] args) {
04.          Integer iOb = 100, iOb2; int n = 99;
05.          iOb2 = m(100); ++iOb; iOb2 = iOb + (iOb / 3);
06.          i = iOb + (iOb / 3); Double dOb = 98.6;
07.          dOb = dOb + iOb; Double valD;
08.          if(b) b = !b; Character ch = 'x';
09.          char ch2 = ch.charValue();
10.      }
11. }
```

[5] *[CSE 20, Final]*

Q17. Consider the following class `Meta`, and its method `myMeth`.

```
01.  import java.lang.annotation.*;
02.  import java.lang.reflect.*;
03.  @Retention(RetentionPolicy.RUNTIME)
04.  @interface MyAnno { String str(); int val(); }
05.  @Retention(RetentionPolicy.RUNTIME)
06.  @interface Meta { String description(); }
07.  @Meta(description = "An annotation test class")
08.  @MyAnno(str = "Meta", val = 99)
09.  class Meta {
10.      @What(description = "An annotation test method")
11.      @MyAnno(str = "Testing", val = 100)
```

```
12.      public static void myMeth() { }
13. }
```

Now write a Java Class named "`AnnotationExam`" with the main method which will print all the annotations of the Meta class and `myMeth` method. The output should look like this:

```
All annotations for Meta:
@What(description="An annotation test class")
@MyAnno(str="Meta", val=99)
All annotations for myMeth:
@What(description="An annotation test method")
@MyAnno(str="Testing", val=100)
```

[5] *[CSE 20, Final]*

Q18. Consider the following class `AnnotationExam`, and its main method.

```
01.  class AnnotationExam {
02.      public static void main(String[] argv) {
03.          try { AnnotationExam} anno= sb.getClass().getAnnotation()
04.              System.out.println("All annotations for Meta:");
05.              for(Annotation a : annos) System.out.println(a);
06.              annos = sb.getClass().getMethod("m").getAnnotations();
07.              System.out.println("All annotations for m:");
08.              for(Annotation a : annos) System.out.println(a);
09.          } catch (NoSuchMethodException exc) {
10.              System.out.println("Method Not Found"); } }
11. }
```

Now write a Java Class named "Meta" and its m method with all the annotations in such a way, that if we run the main method in AnnotationExam the output should look like this:

```
All annotations for Meta:
@What(description="An annotation test class")
@MyAnno(str="Meta", val=50)
All annotations for m:
@What(description="An annotation test method")
@MyAnno(str="Method M", val=25)
```

[5] *[CSE 21, Final]*

Q19. Consider the following class `AnnotationExam`, and its main method.

```
01.  class AnnotationExam {
02.      public static void main(String[] argv) throws NoSuchMethodException {
03.          Annotation[] annos = Alpha.class.getAnnotations();
04.          System.out.println("All annotations for Alpha:");
05.          for(Annotation a : annos) System.out.println(a);
06.          annos = Alpha.class.getMethod("m").getAnnotations();
07.          System.out.println("All annotations for m:");
08.          for(Annotation a : annos) System.out.println(a);
09.      }
10. }
```

Now write a Java Class named "**Alpha**" and its m method with all the annotations in such a way, that if we run the main method in AnnotationExam the output should look like this:

```
All annotations for Alpha:
@A2(s2="Class Alpha")
@A1(s1="Alpha A1", n1=55)
All annotations for m:
@A2(s2="Alpha method M")
@A1(s1="Alpha M A1", n1=45)
@A3()
```

[5] *[CSE 22, Final]*

Q20. Consider the class `AnnotationExam` provided. Your task is to: Reverse-engineer the Java Annotations. Given the usage in class **Alpha** and the resulting output, provide the full definitions for the custom annotations **@A1, @A2 and @A3** (with proper *Retention* and *Target* policies).

```
1.  @A2(s2 = "Class Alpha")
2.  @A1(n1 = 101)
3.  public class Alpha {
4.      @A2(s2 = "Alpha method M")
5.      @A1(s1 = "Alpha M", n1 = 200)
6.      @A3(100)
7.      public void m() {}
8.  }
1.  import java.lang.annotation.*;
2.  import java.lang.reflect.*;
3.  public class AnnotationExam {
4.      public static void main(String[] args) throws Exception {
5.          printAnnos(Alpha.class, "Class Alpha");
6.          printAnnos(Alpha.class.getMethod("m"), "Method m");
7.      }
8.      static void printAnnos(AnnotatedElement e, String label) {
9.          10.println(label + " - ");
10.         for(Annotation a : e.getAnnotations()) 10.println(a);
11.     }
12. }
```

Target Output:

```
Alpha Class:
@A2(s2 = "Class Alpha")
@A1(s1 = "Final", n1=101)
Method m:
@A2(s2 = "Alpha method M")
@A1(s1 = "Alpha M", n1=200)
@A3(100)
```

[5] *[CSE 23, Final]*

11. Lambda Expressions, Modules, Records

Q1. Write a `SAM` as a target for the lambda expression: `n -> (n%2 == 0)`. **[2]** *[CSE 20, Final]*

Q2. Write a `SAM` as a target for the lambda expression: `n -> n*2.0`. **[2]** *[CSE 21, Final]*

Q3. What is the purpose of using lambda expressions in Java? Write a block lambda expression for finding the factorial of an integer number n. **[5]** *[CSE 21, Final]*

Q4. What is the difference between `equals()` method and equality operator (==) in Java? Show with a code example. **[5]** *[CSE 21, Final]*

Q5. Write the output of the following Java program.

```
01.  interface MyFunc<T> { T func(T v); }
02.  class MyArrayOps {
03.      static <T> T val(T v) { return v; }
04.  }
05.  public class LambdaExam {
06.      static <T> MyOp(MyFunc<T> f, T v) { return f.func(v); }
07.      public static void main(String[] args) { int i = 10;
08.          MyFunc<String> d = str -> str+str;
09.          System.out.println(d.func("Lambda"));
10.          MyFunc<Integer>r=(n)->{n=n+1;return n*n*i;};
11.          System.out.println(f.func(8));
12.          var value = MyOp(MyArrayOps::<Integer>val, 4);
13.          System.out.println(value);
14.          System.out.println(MyOp(MyArrayOps::<String>val, "Two"));
15.      }
16. }
```

[5] *[CSE 20, Final]*

Q6. Trace the following Java program and write the exact output.

```
01.  interface I<T> { T f(T i); }
02.  class A { static int a(int i) { return i * 2 - 5; }
03.      static String s(String s) { return s + s.getClass(); }
04. }
05.  public class LambdaExam {
06.      static <T> T run(I<T> k, T v) { return k.f(v); }
07.      public static void main(String[] args) {
08.          I<String> s1 = s -> "[" + s + "]"; 10.println(s1.f("java"));
09.          I<Integer> s2 = n -> {
10.              10.println(n % 5 == 0 ? "Even" : "Odd"); return n * n;
11.          };
12.          10.println(s2.f(7));
13.          10.println(A::a, 10);
14.          10.println(A::s, "nut");
15.      }
16. }
```

[5] *[CSE 23, Final]*

12. String Handling & Collections Framework

Most String/Collections content appears alongside related topics (Data Types, Inheritance, Classes). See:

- Data Types & Strings – "StringDemo" program output (CSE 16-17, Final).
- Classes & Methods – ArrayList duplicate removal (2017-18 Akash).
- Inheritance – GUI program using ArrayList (CSE 16-17, Final).

13. AWT, Swing, Event Handling

Swing/AWT content appears in:

- Inheritance – JFrame Box GUI program (CSE 16-17, Final).
- Classes & Methods – Listener vs Adapter (CSE 16-17, Final).
- Packages & Interfaces – Default layout manager True/False (CSE 16-17, Final).